Qwen

From Sphere to Substrate: Adapting Graph-Based Terrain Algorithms for Trixel's Geometry-Agnostic Architecture

Deconstructing the Core Architecture: A Graph-Centric Simulation Model

The analysis of the `OptimisticPeach/sphere_terrain` repository reveals a fundamental architectural pattern that is highly relevant to the goals of the Trixel engine. The project is not merely a procedural terrain generator; it is a sophisticated simulation engine that operates on a graph-based representation of a surface [4]. Its core design philosophy is the strict separation of terrain computation from geometric representation, a principle that aligns directly with the existing doctrine of Trixel, which distinguishes between source adjacency, compiled adjacency, and geometric proximity [4]. This architectural choice makes `sphere_terrain` an ideal case study for harvesting reusable, geometry-agnostic algorithms. The entire workflow can be abstracted into a clear sequence of operations performed on a topological graph, independent of the underlying mesh's final shape.

The foundation of this graph-centric model is provided by the `hexasphere` library, which serves as the base topology provider [1] [15]. This library specializes in subdividing base shapes, such as an icosahedron, into a dual Goldberg polyhedron [3]. The resulting mesh is composed exclusively of hexagonal and pentagonal faces, providing a robust and nearly uniform grid structure suitable for world-building [3] [10]. The most critical feature of `hexasphere` for this research is its ability to generate an explicit adjacency map from the indices produced during the subdivision process [3] [15]. This capability transforms the raw vertex and face data into a structured graph where each cell (hexagon or pentagon) has a well-defined list of its immediate neighbors. This adjacency information is the key enabler for all subsequent graph-based operations within `sphere_terrain`. The library also provides mechanisms for identifying cells using a coordinate system based on their position relative to the original icosahedron's faces and edges, allowing for precise lookups and neighbor computations using vector offsets [10].

Once this foundational topology is established, `sphere_terrain` proceeds to attach a rich set of stateful properties to each node in the graph. These properties represent the various environmental fields that constitute the terrain. The primary field is elevation, which assigns a scalar value to each cell, representing its height above a given datum [4]. Beyond elevation, the simulation generates hydrological states, including wetness and flow, which are used to simulate water movement and deposition [4]. Finally, a biome classification is derived based on the culmination of these physical and hydrological states, assigning a semantic label like 'Desert' or 'Tundra' to each region of the landscape [4]. The crucial insight is that `sphere_terrain` does not perform any rendering itself; its sole responsibility is to compute these state fields across the graph [4]. It produces data structures containing the computed values, leaving the task of converting these values into a visual mesh to the consumer application [4]. This clean separation of concerns is the single most valuable aspect of the project for the Trixel research goal. It demonstrates a complete pipeline from a topological abstraction to a concrete data representation ready for interpretation by another system.

To facilitate navigation and manipulation of this graph, the architecture relies on specific Rust constructs. While a full inventory requires direct code inspection, preliminary analysis points to several key structures and modules that serve as navigation landmarks. The `hexasphere` crate provides the core abstractions, likely centered around a `Subdivided<T, S>` struct that takes a base shape (`S`) and applies a subdivision algorithm [15]. The `BaseShape` trait would define the contract for different initial geometries, such as an Icosahedron or Tetrahedron [3] [15]. For adjacency, the `hexasphere::adjacency` module is paramount, as it is designed to create neighbor maps from the indices generated by the `Subdivided` struct [15]. In `sphere_terrain`, there would logically be a central data structure representing a single node or cell in the graph. This structure would contain the computed state fields: elevation, wetness, river status, etc., along with a reference or identifier to its list of neighbors obtained from the adjacency map. Operations like erosion would then iterate over this collection of nodes, accessing their neighbor lists to perform calculations. The documentation notes that parallelization is achieved using the Rayon library with atomic instructions, indicating that the core simulation loop is designed to operate on multiple nodes concurrently [4]. However, as per research directives, the specifics of this concurrency model are considered implementation-specific and will not be a focus of this report.

The overall architecture can be summarized as a state machine operating on a graph. The initial state is defined by the topology (the hexasphere grid). The simulation then iteratively updates the state of each node based on its own properties and the properties of its neighbors. The final output is a fully realized graph where every node carries a

comprehensive set of terrain-related data. This modular approach allows for distinct systems to be developed and tested independently. For instance, the base height generation algorithm can be swapped out for a different one, or the erosion model can be replaced, without affecting the core graph traversal mechanism. This modularity is a hallmark of good software design and is precisely the kind of flexible system required to build a versatile terrain substrate for Trixel. The `sphere_terrain` project successfully demonstrates that complex geological processes can be modeled effectively on a discrete, polygonal graph, providing a powerful template for developing a geometry-agnostic terrain engine.

Algorithmic Principles of Terrain State Manipulation

The true value of `sphere_terrain` lies not in its final rendered output but in the abstract principles it employs to manipulate terrain state across a neighbor graph. By dissecting its algorithms, we can identify a universal pattern for graph-based terrain operations: `terrain state → graph/node → known neighbors → terrain operation → updated state`. This section analyzes the core algorithms for height generation, wetness propagation, river construction, and biome derivation, explicitly categorizing their assumptions as GEOMETRY-AGNOSTIC, SPHERE-DEPENDENT, or IMPLEMENTATION-SPECIFIC. This classification is critical for isolating the reusable logic from the spherical context in which it was originally implemented.

Height Generation: The initial elevation field is the bedrock upon which all other processes are built. The `sphere_terrain` project utilizes simplex noise for this purpose, leveraging Ralith's `clatter` library, which is noted to require the Rust nightly compiler [4]. The process begins by generating a base height map using this noise function. To add more realism, the author of a similar implementation describes a pre-processing step that enhances the noise field by considering distance to coastlines and applying slope-dependent amplitude modulation [5]. This means that subsequent octaves of noise have greater influence in steeper regions, controlled by a configurable curve [5]. A key part of this algorithm involves calculating the shortest distance from each point to the coast, applying a configurable curve to this distance value, and using it as a modifier for the base height [5]. This entire process, however, is fundamentally tied to the concept of a "coast," which implies a spherical world. Therefore, while the *mechanism* of using a noise function to seed an elevation field is general, the *logic* for coastal modification is SPHERE-DEPENDENT. The core assumption is that a global ocean exists, and distance to it can be meaningfully calculated. The geometry-agnostic version for Trixel would need

to replace this with a more abstract concept, perhaps based on a user-defined starting boundary or a simple gradient towards a central point. The implementation detail of using the `clatter` library is, of course, implementation-specific.

Hydraulic Erosion: As detailed in the next section, `sphere_terrain` implements a droplet-based hydraulic erosion algorithm [5]. This method is inherently suited for a graph-based approach. In each simulation step, a particle ("droplet" or "snowball") is spawned in every cell simultaneously [5]. Each particle possesses a state comprising its position, speed, water volume, and sediment load [5] [6]. The core logic is executed in parallel: for each droplet, the algorithm finds the lowest neighboring node, moves the droplet to that location, and then updates the terrain at the droplet's previous position [5] [6]. If the droplet's sediment load exceeds its carrying capacity (a function of its speed and travel distance), it deposits some material; if it has less than its maximum capacity, it erodes material from the ground [5] [6]. This entire process is a prime example of a GEOMETRY-AGNOSTIC algorithm. It only requires the ability to find a neighbor with a minimum property value (elevation) and move a state object across that edge. The calculation of carrying capacity and deposition/erosion rates can be purely numerical formulas, independent of any underlying Cartesian coordinates. The only potential sphere-dependency could arise in the definition of "slope." While the basic droplet logic only compares elevations of adjacent nodes, a more advanced implementation might calculate velocity based on a continuous surface normal, which would require vector math tied to the spherical geometry [6]. For Trixel, this would need to be simplified to a function of the elevation difference between the current and next node.

Wetness Propagation and River Construction: These two phenomena are closely related and often emerge naturally from the same simulation loop. Wetness can be conceptualized as a saturation level within each cell, accumulating "rain" over time and draining to lower neighbors [12]. This creates a simple cellular automaton rule: in each step, each cell distributes its excess moisture to its lowest neighbor. This logic is entirely GEOMETRY-AGNOSTIC, relying only on the adjacency list and a comparison of node properties. River construction is typically a post-processing or concurrent step that identifies paths of high cumulative flow. In the droplet-based model, areas where many simulated droplets converge after traveling significant distances indicate the formation of valleys and rivers [6]. This is achieved by maintaining a separate "traffic" or "erosion/deposition" texture or field that tracks the total amount of activity at each node [6]. Again, this is a GEOMETRY-AGNOSTIC process. It involves aggregating data (flow counts) from particles arriving at a node, regardless of the path they took. The final river path can be traced by following the flow from high-accumulation nodes to lower ones.

The sphere-dependency here is minimal, limited perhaps to defining the initial rainfall distribution, which might be based on latitude on a sphere.

Biome Derivation: The final step in the simulation is to translate the continuous physical and hydrological fields into discrete semantic classes. This is done through a series of rules that map combinations of elevation, wetness, temperature, and other factors to a biome type [4]. For example, a rule might be: "IF elevation > 0.7 AND wetness < 0.3 THEN biome = 'Alpine Tundra'" [4]. This process is a straightforward lookup or decision tree based on the final state of the graph nodes. The logic itself is completely GEOMETRY-AGNOSTIC. It is a pure function of the node's properties. The only sphere-dependent aspect would be the input parameters themselves. For instance, a parameter like "latitude" might be used to determine temperature, which in turn affects biome choice. In a Trixel context, these parameters would need to be redefined in a geometry-agnostic manner, perhaps using distance from a user-defined pole or a procedural temperature map. The core algorithm—mapping a multi-field state to a discrete class—is universally applicable.

Algorithm Component	Core Mechanism	Key Data Flow Across Edges	GEOMETRY-AGNOSTIC?	Assumed Dependencies
Height Generation	Procedural noise functions seeded with a deterministic value.	None. Computed locally at each node.	Partially	Requires a concept of "distance to coast" or a global boundary.
Hydraulic Erosion	Particle simulation (droplets/ snowballs) moving downhill, eroding and depositing sediment.	Particle state (speed, water, sediment) moves to the lowest neighbor.	Yes	Slope calculation for velocity may depend on surface normals.
Wetness Propagation	Cellular automaton where moisture drains from each cell to its lowest neighbor.	Moisture/water volume flows to the lowest neighbor.	Yes	Assumes a consistent drainage direction based on elevation.
River Construction	Tracking cumulative particle flow or erosion/deposition activity at each node.	Information about flow/ rainfall accumulates at the destination node.	Yes	Depends on the output of the erosion/hydrology simulation.
Biome Derivation	Rule-based decision tree or lookup table mapping terrain states to semantic classes.	None. Operates on final, computed node states.	Yes	Biome thresholds and dependencies (e.g., temperature) may be sphere-dependent.

This analysis demonstrates that the vast majority of the computational logic in `sphere_terrain` is encapsulated within a graph traversal framework. The operations are local, depending only on a node's own state and the states of its immediate neighbors. This locality is what makes the algorithms so powerful and adaptable. They do not require global information about the entire world (beyond the local neighborhood), making them perfectly suited for a system like Trixel that needs to operate on arbitrary polygonal fields, whether flat, curved, or of an irregular manifold.

Hydraulic Erosion on a Neighbor Graph: An Abstraction Analysis

Hydraulic erosion is a cornerstone of realistic terrain generation, and its implementation in `sphere_terrain` offers a particularly insightful case study in abstracting physics-based simulation onto a discrete graph. The project's chosen method appears to be a variant of the particle-based droplet or "Blurosion" algorithm, a technique widely recognized for its effectiveness in creating branching valley networks and its suitability for parallel processing [5] [6]. Unlike grid-based methods that solve partial differential equations over a continuous domain, the droplet model simulates individual particles of water, making it conceptually closer to real-world physics and easier to adapt to non-grid topologies. This section provides a deep abstraction of the hydraulic erosion algorithm, focusing on its constituent parts and classifying them according to their dependency on the underlying geometry.

The fundamental unit of the simulation is the "droplet" or "particle." In the `sphere_terrain`-inspired model, a single droplet is simulated within each cell of the graph simultaneously [5]. Each droplet maintains a small state vector, which is the essence of its identity during the simulation. This state typically includes its current position (a pointer or ID to a graph node), its speed, its volume of water, and its current sediment load [5] [6]. This encapsulation of state is crucial; it allows the simulation to be broken down into a series of discrete, parallelizable steps. The simulation proceeds in discrete time steps, where each droplet's state is updated based on local conditions.

The first step in each time step is **neighbor traversal and movement**. The droplet's logic dictates that it must move downhill [5]. On a rectangular grid, this is often determined by comparing the height of the current cell to its four (or eight) orthogonal/diagonal neighbors. On the hexasphere graph from `sphere_terrain`, this logic is identical in principle but differs in execution. The droplet examines its list of known neighbors, which is provided by the `hexasphere` adjacency map [2] [15]. It then iterates through this list to find the single neighbor with the lowest elevation value [5]. This selection process is purely a comparison of a single scalar property (elevation) among connected nodes. This is a quintessential GEOMETRY-AGNOSTIC operation. It requires no knowledge of Cartesian coordinates, angles, or distances; it only requires an ordered relationship (less than/greater than) on the elevation field. Once the lowest neighbor is identified, the droplet moves exactly one step across the graph edge to occupy that new node [5]. This "move one step" rule is simple, deterministic, and completely independent of the graph's embedding in 3D space. It forms the core of the graph traversal mechanism.

The second step is **interaction and state update** at the destination node. After moving, the droplet interacts with the terrain at its *previous* location—the node it just left [6]. This is a subtle but important detail that prevents artifacts like self-digging. The interaction is governed by two primary rules based on the droplet's sediment carrying capacity. The carrying capacity is typically a function of the droplet's speed and volume of water [5] [6]. First, if the droplet's current sediment load is less than its capacity, it has the potential to pick up more material. It does so by lowering the elevation of its previous position, thereby eroding the terrain [5]. Second, if the droplet's sediment load exceeds its capacity, it must deposit some of that sediment at its previous location, raising the elevation there [5]. These actions are performed as local modifications to the elevation field of the node being departed. The change in elevation is a function of the droplet's state (its speed, size, and sediment load) and configurable constants that govern erosion and deposition rates [6]. The formulas governing these interactions are numerical recipes, not geometric calculations, and are therefore GEOMETRY-AGNOSTIC.

The third step is **state update at the destination node**. Upon arriving at its new home, the droplet's state is also updated. Its speed may change based on the slope of the new terrain, and its water volume might decrease due to seepage or evaporation. Crucially, if multiple droplets arrive at the same node during a single time step, their effects are merged [5]. This aggregation is a form of accumulation that builds up flow and sediment load as paths converge, which is essential for carving deep channels. The merging process is again a local operation, combining the state vectors of particles arriving at a common destination node.

Throughout this entire process, there are very few assumptions that are strictly SPHERE-DEPENDENT. The most likely candidate is the calculation of the droplet's velocity. While the basic movement rule is simply "go to the lowest neighbor," a more sophisticated model might calculate velocity based on the gravitational potential energy difference, which would involve the angle of the slope. On a sphere, this angle is derived from the dot product of surface normals, which are defined by the 3D positions of the vertices [6]. A geometry-agnostic version for Trixel would need to simplify this. Instead of a continuous normal, the velocity could be a direct function of the elevation difference between the current and next node, scaled by a constant. This would lose some physical accuracy but retain the essential behavior of faster flow on steeper slopes without needing spherical math. Another minor dependency might be in the initialization phase, where the droplet spawning rate (rainfall) could be modulated by latitude, a concept unique to spheres [4]. For Trixel, rainfall would need to be defined via a different spatial function, such as distance from a central point or a procedural wind-driven pattern.

The implementation-specific details, which fall outside the scope of this research, include the use of the `clatter` library for noise generation [4], the use of Rayon for parallelization [4] [5], and the specific data structures used to manage the droplets (e.g., a 2D array aligned with a heightmap, though this would need adaptation for a general graph) [5]. The core algorithm, however, is a beautiful example of a physics simulation distilled down to its essential graph-theoretic components: a stateful particle traverses a graph by finding a minimum-weight neighbor and updates the graph's properties locally. This abstraction is powerful because it can be applied to any graph structure, whether it represents a sphere, a flat plane, a cylinder, or an irregular manifold, making it an ideal candidate for integration into the Trixel engine.

Structural Success Criteria and Measurement Protocol for Proof 3

Proof 3, "Elevation over Curvature," is the pivotal experiment designed to validate whether the graph-based terrain algorithms harvested from `sphere_terrain` can produce a meaningful geological feature on a non-spherical, curved polygonal field. The objective is not to achieve photorealism but to confirm that the underlying simulation mechanics are sound and preserve topological integrity. The following criteria and measurement protocol are defined structurally, establishing a clear benchmark for success without prescribing arbitrary numeric tolerances, which will be determined during the subsequent implementation phase.

A successful proof must demonstrate that the application of an elevation field to a curved polygonal mesh results in a coherent, artifact-free geological feature. This requires satisfying a set of structural invariants that ensure the simulation behaves predictably and logically. These criteria form a checklist for evaluating the outcome of the experiment.

Structural Success Criteria for Proof 3:

- **A. Acceptance of Variation:** The polygonal field must correctly accept and store a non-uniform elevation field. This is the most basic requirement; the system must be able to represent hills, ridges, and valleys, not just a flat plane. The elevation data must coalesce across adjacent cells to form a coherent geological feature, such as a distinct ridge or valley. This tests that the elevation values are influencing the overall topology meaningfully.

- **B. Identifiable Feature Formation:** The elevation data must coalesce across adjacent cells to form a coherent geological feature, such as a distinct ridge or valley. This tests that the elevation values are influencing the overall topology meaningfully.
- **C. Surface Connectedness:** The resulting surface must remain a single, continuous entity. No part of the mesh should become disconnected from the rest. This ensures that the elevation function does not introduce unintended fractures or tears in the geometry.
- **D. No Undeclared Geometric Artifacts:** This is a paramount criterion. The process of elevating the mesh must not produce cracks, gaps, overlaps, or T-junctions between neighboring cells. The triangulation or meshing algorithm used to render the result must produce a valid, watertight mesh [8].
- **E. Adjacency Preservation:** The fundamental adjacency relationships declared in the initial topology must persist. If Cell A was adjacent to Cell B before elevation, they must still share an edge afterward. The declaration of neighbors is a topological fact that must not be violated by the geometric transformation.
- **F. No Spurious Neighbor Creation:** The elevation process must not cause two previously non-adjacent cells to become neighbors or intersect. The topology of the graph is fixed; the simulation cannot alter it by creating new connections.
- **G. No Topological Surgery:** The process must not require undeclared modifications like splitting, merging, or deleting cells. The number of cells/facets should remain constant throughout the operation. The simulation should work with the given topology, not on it.
- **H. Validity Across Orientation:** The method must work regardless of the face orientation. A successful test would involve creating a feature that crosses from one face of the icosahedron to another, proving it works on genuinely curved surfaces, not just a flattened net.
- **I. Determinism:** Given the exact same inputs (topology, seed, elevation function), the output must be identical every time. This ensures the process is predictable and debuggable.
- **J. State-Rendering Separation:** The elevation values themselves must remain identifiable and separable from the rendered geometry. One should be able to query a cell's elevation without needing to inspect the mesh. This confirms that the terrain state is distinct from its geometric embodiment.

Measurement Protocol for Implementation:

While acceptable tolerance ranges for these metrics will be established during implementation testing, the research document must specify exactly what data to collect.

This quantitative data will be essential for later analysis and for setting those definitive thresholds.

The following measurements should be recorded during the execution of Proof 3:

Metric Category	Metric Name	Description	Measurement Method
Geometric Metrics	Edge Length Change Ratio	The ratio of the length of an edge after elevation to its length before.	Compute lengths using 3D coordinates of vertices. Track min, max, and average ratios.
	Face Area Change Ratio	The ratio of the area of a face after elevation to its area before.	Compute areas of triangular faces. Track min, max, and average ratios.
	Aspect Ratio Change	The change in the aspect ratio of triangular faces to detect shearing.	Calculate aspect ratios (e.g., ratio of longest to shortest side) before and after.
Topological Metrics	Neighbor Count Stability	Verifies that the number of neighbors for each node remains unchanged.	Count neighbors for each node in the graph before and after elevation.
	Crack/Gap Distance	The minimum distance between any two adjacent faces.	For each shared edge, measure the distance between the corresponding line segments in 3D space.
	Overlap/ Intersection Count	The number of times non-adjacent faces intersect each other.	Perform pairwise intersection tests between all non-adjacent faces.
Physical/ Simulation Metrics	Normal Angle Change	The angular deviation of surface normals at cell centroids.	Calculate the angle between the normal vector before and after elevation. Track changes.
	Dihedral Angle Change	The angle between the planes of two adjacent faces.	Calculate the dihedral angle for each edge before and after elevation.
Deterministic Verification	Deterministic Output Hash	A cryptographic hash of the final elevation field to ensure bitwise reproducibility.	Generate a SHA-256 or similar hash from the serialized elevation values of all nodes.

By adhering to this protocol, the engineering team executing Proof 3 will produce a dataset that quantitatively validates the structural success criteria. For example, a successful run would show a near-zero crack/gap distance, a zero overlap count, stable neighbor counts, and a consistent output hash when rerun. The geometric metrics (edge length, area, etc.) will provide the necessary data to establish what constitutes an "acceptable" amount of distortion for the Trixel engine. This systematic approach transforms the qualitative goal of "seeing if it works" into a rigorous scientific experiment with clear pass/fail conditions and actionable data.

Conceptual Integration: Candidate Mappings to the Trixel Semantic System

The ultimate goal of this research is not to replicate `sphere_terrain` but to leverage its insights to inform the development of the Trixel engine. A critical step in this process is to create a conceptual bridge between the abstract outputs of `sphere_terrain`'s simulation and the existing Trixel semantic recipe system. This mapping should be presented as a non-binding, unproven set of candidate seams, intended to spark discussion about how a scientifically grounded terrain substrate could inform, rather than replace, the art and material decisions made by Trixel recipes. This approach preserves architectural flexibility and avoids forcing an immediate, potentially suboptimal, integration path.

First, we must abstract the output of `sphere_terrain` into its core, geometry-independent components. The project's simulation produces a graph where each node (cell) holds a rich set of stateful properties. These outputs are largely decoupled from the spherical geometry, making them highly portable concepts. The primary data entities generated are:

- **Node Identity:** A unique identifier for each cell in the graph, forming the basis of the topology.
- **Neighbor Relationships:** An explicit adjacency list defining the topological connections between cells.
- **Position:** The 3D coordinates of the cell's centroid or vertices. This is the component most dependent on the spherical geometry and would need to be replaced by Trixel's own geometric authority.
- **Elevation:** A scalar value representing the height of the terrain at that node.
- **Hydrological State:** Fields for `flow`, `wetness`, `river_information`, and potentially `deposition_rate`.
- **Biome Classification:** A discrete semantic label (e.g., 'Temperate Forest', 'Savanna') assigned to the node based on its physical and hydrological state.

With this abstracted output in mind, we can now explore potential mappings to Trixel's architecture. The key insight is that `sphere_terrain`'s simulation would act as a "terrain substrate" or "geography generator," producing a factual, stateful description of the land. Trixel recipes would then consume this state to make artistic and material decisions. This reframes the role of recipes from creators of geography to interpreters of it.

The following table presents a non-binding candidate mapping. Each arrow signifies a potential connection, not a mandatory integration path. The rationale explains how this mapping could enhance the Trixel engine's capabilities by enforcing a clearer separation between simulation and art.

`sphere_terrain` Output	Geometry-Agnostic?	Candidate Trixel Mapping	Rationale
Node Identity / Position	Partially	**Trixel Compiled Adjacency:** Used to build the compiled graph. **Geometry Authority:** Provides the initial anchor points for mesh generation.	The core topology is reusable. The position data would feed into Trixel's own geometry system, which would handle the actual vertex placement.
Elevation	Yes	**World/Cell Elevation Authority:** Directly becomes the source for Trixel's elevation data.	This is the most straightforward mapping. The generated elevation field replaces procedural generation within Trixel's world-building pipeline, providing a physically plausible base.
Biome Classification	Yes	**Trixel Recipe Selection:** Acts as a primary input for selecting a base terrain recipe.	Instead of recipes inventing geography, they consume the geographic reality produced by the substrate. A 'Desert' biome node could trigger the 'Sandy Desert' recipe.
Wetness / Moisture	Yes	**Recipe Parameter Input / Material Selection:** Influences parameters like 'slope' or 'moisture' in a recipe, or selects between different material variants.	Allows recipes to react to environmental conditions. For example, a 'Coastal Grassland' recipe could use the wetness value to decide between sandy and muddy textures.
River Information	Yes	**Topology/Terrain Feature Request:** Could trigger a special "riverbed" feature in the Trixel geometry system.	Enables the creation of features that are semantically distinct from regular terrain. The substrate identifies the river, and the geometry system carves the channel.
Neighbor Graph	Yes	**Trixel Compiled Terrain Adjacency:** This is the foundation of the Trixel graph.	Confirms that the core graph structure is compatible and can be used directly by Trixel.

This mapping suggests a significant evolution of the Trixel recipe system. Currently, recipes may be responsible for generating the underlying landscape through ordered passes of noise, gradients, and blending [4]. This research indicates that recipes could instead evolve into an *art interpretation layer* or a *material selection layer* that operates on a scientifically generated terrain substrate. The substrate would answer the question, "What is the geography?" (Is it a mountain? A valley? How wet is it?). The recipes would then answer, "Given this geography, what materials and art should be used to represent it?" This aligns perfectly with the user's vision of connecting "polygon geometry generates the physical land" with "Trixel recipes decide what that land looks like."

It is crucial to recognize that this is a thought experiment. The actual implementation within Trixel may differ. For example, the `BiomeClassification` might not directly select a recipe but instead become a parameter that influences the blending weights between several candidate recipes. Similarly, `RiverInformation` might not request a

special feature but could simply provide a strong directional signal to a recipe that carves its own channels. The purpose of this document is to provide a vocabulary and a conceptual framework for discussing these possibilities. By defining these candidate seams, we create a clear path forward for experimentation and development, grounded in the proven principles of a working, graph-based terrain simulation.

Reference

1. hexasphere - crates.io: Rust Package Registry https://crates.io/crates/hexasphere
2. OptimisticPeach/rehexed: Adjacency lists for hexasphere's ... https://github.com/OptimisticPeach/rehexed
3. OptimisticPeach/hexasphere: Subdividing triangle meshes. https://github.com/OptimisticPeach/hexasphere
4. Hydraulic Erosion Simulation on a Sphere ... https://github.com/OptimisticPeach/sphere_terrain
5. Around The World, Part 23: Hydraulic erosion https://frozenfractal.com/blog/2025/6/6/around-the-world-23-hydraulic-erosion/
6. Simulating hydraulic erosion https://jobtalle.com/simulating_hydraulic_erosion.html
7. Voronoi-Based Procedural Terrain Generation with Hydrology ... https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SoCG.2026.101
8. Structured Bitmap-to-Mesh triangulation for geometry- ... https://www.sciencedirect.com/science/article/pii/S152407032600007X
9. tessapower/hydraulic-erosion https://github.com/tessapower/hydraulic-erosion
10. AllphinsPilot/hexasphere: A module to create hexagonal ... https://github.com/AllphinsPilot/hexasphere
11. [2210.14496] Visually Improved Erosion Algorithm for the ... https://arxiv.org/abs/2210.14496
12. An Efficient Data Structure for 3D Multi-Layer Terrain and ... https://nickmcd.me/2022/04/15/soilmachine/
13. SoilMachine: Multi-Layer Realtime Erosion System [WIP] https://www.reddit.com/r/proceduralgeneration/comments/nsz1wp/soilmachine_multilayer_realtime_erosion_system_wip/

14. Visually Improved Erosion Algorithm for the Procedural ... https://www.researchgate.net/publication/364771642_Visually_Improved_Erosion_Algorithm_for_the_Procedural_Generation_of_Tile-based_Terrain
15. hexasphere - Rust https://doc.qu1x.dev/bevy_trackball/hexasphere/index.html